

Lab 3: Architecture Selection Justification

Hyperlocal Courier Dispatch & Tracking Engine (Problem Statement #23)

Meghana Grandhi | PES1UG24CS268 | Section E

Architecture Selection: We chose Microservices Architecture for the Hyperlocal Courier Dispatch & Tracking Engine.

Architectural Choice

The system is split into small, independently deployable services behind a single API Gateway: Courier Request, Dispatch & Matching, Tracking, Routing, Delivery Verification and Notification, with an Order Database and the external Routing / GPS Service. The services are organised around the project's requirements and use cases (FR-001 to FR-005, NFR-001). The Layered and Client-Server styles were also considered:

Style	Fit for this scenario	Main drawback here
Layered	Simple and familiar, but all features scale together.	Cannot scale live tracking on its own; one fault can affect every layer.
Client-Server	Easy central control and data consistency.	Single point of failure; weak against the 99.5% availability target.
Microservices (chosen)	Each feature runs and scales as its own service.	More operational complexity and network calls between services.

Two Reasons

1. Different parts of the system have very different loads. The Tracking Service receives continuous GPS updates from every active rider, while request creation and OTP verification happen only occasionally. With microservices, Tracking can be scaled on its own to keep updates under 2 seconds (NFR-001) without scaling the whole system.

2. A failure stays contained, which supports 99.5% availability. If the Routing or Notification service goes down, riders can still be matched and deliveries can still be completed with the OTP. In a single-server Client-Server design the same fault would stop the entire platform (NFR-002).

Security Advantage

Services are not exposed directly. All Sender and Rider traffic enters through the API Gateway, which authenticates the user before forwarding the request. OTP checking exists only in the Delivery Verification Service, so no other component can mark a parcel as Delivered without a correct OTP (FR-005).

Performance Benefit

Heavy work is isolated. CPU-intensive route optimization (FR-004) runs in the Routing Service and cannot slow down the Tracking Service. Tracking can use a lightweight push channel (WebSocket) to send live location and status to the Sender Client quickly.

Trade-off

Microservices add network calls and operational complexity. We keep the number of services small (one per core capability) and use simple REST interfaces to limit this overhead.